

Welcome to SEAS Online at George Washington University

Class will begin shortly

Audio: To eliminate background noise, please be sure your audio is muted. To speak, please click the hand icon at the bottom of your screen (Raise Hand).

When instructor calls on you, click microphone icon to unmute. When you've finished speaking, *be sure to mute yourself again.*

Chat: Please type your questions in Chat.

Recordings: As part of the educational support for students, we provide downloadable recordings of each class session to be used exclusively by registered students in that particular class for their own private use.

Releasing these recordings is strictly prohibited.

SEAS 8515

Data Engineering for AI

Professor Steven Cocke
Week 5: June 13, 2026

Agenda

- Admin Items
- Week 4 Assignment Review
- Data Ingestion & Streaming Concepts
- APIs
- Assignment Overview for This Week

Admin Items

- Week 5 (today) material will be on midterm
- Week 6 (next week) material will not be on midterm
- There is a homework assignment this week
- There will be no homework assignment next week (week 6)
- There will be a homework after the midterm (week 7 going into week 8 lecture)
- Practice test is not worth any points
 - Live on Blackboard

THE GEORGE
WASHINGTON
UNIVERSITY

WASHINGTON, DC

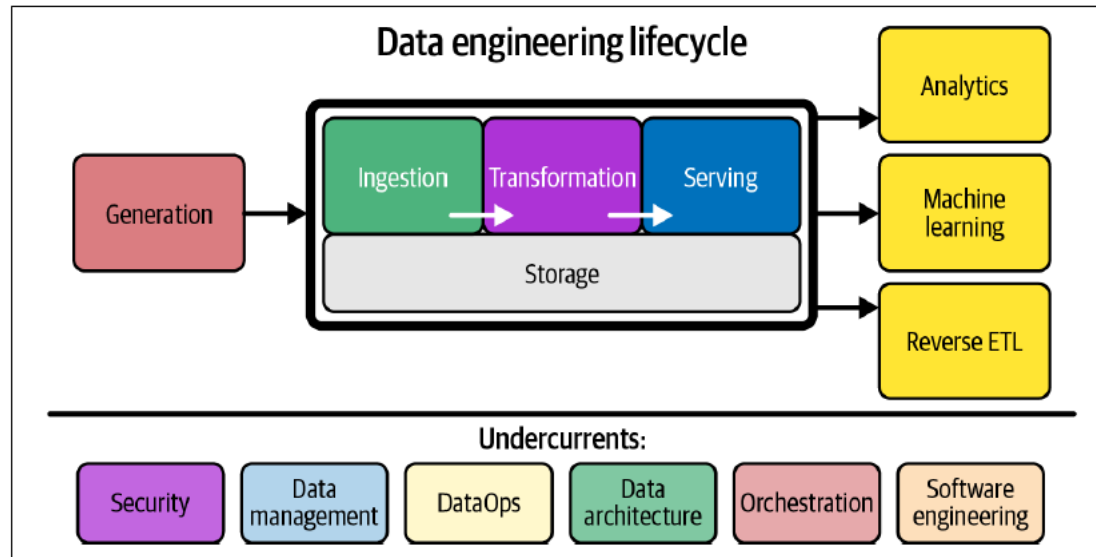
Week 4 Assignment Overview

THE GEORGE
WASHINGTON
UNIVERSITY

WASHINGTON, DC

Data Ingestion & Streaming Concepts

Data Engineering Lifecycle



- Generation
- Storage
- Ingestion
- Transformation
- Serving

Src: Fundamentals of Data Engineering (Reis & Housley)

How is Data Created?

Data is an unorganized, context-less collection of facts and figures. It can be created in many ways, both analog and digital.

- Analog data creation occurs in the real world, such as vocal speech, sign language, writing on paper, or playing an instrument. This analog data is often transient.

How is Data Created?

- Digital data is either created by converting analog data to digital form or is the native product of a digital system.
- Examples:
 - Analog to digital: is a mobile texting app that converts analog speech into digital text.
 - Digital data creation is a credit card transaction on an ecommerce platform. A customer places an order, the transaction is charged to their credit card, and the information for the transaction is saved to various databases.

Files & Unstructured Data

- A file is a sequence of bytes, typically stored on a disk (store local parameters, events, logs, images, and audio)
- Files are a universal medium of data exchange.
- Much of the world still sends and receives files (e.g. government agencies), there's an excellent chance you'll download the data as an Excel or CSV file or receive the file in an email.

The main types of source file formats:

- Excel, CSV, TXT, JSON, and XML. These files can be structured (Excel, CSV), semistructured (JSON, XML, CSV), or unstructured (TXT, CSV).
- Parquet

Application Databases (OLTP)

- An application database stores the state of an application (e.g. database that stores account balances for bank accounts.)
- As customer transactions and payments happen, the application updates bank account balances.
- Typically, an application database is an online transaction processing (OLTP) system (Reads and writes individual data records at a high rate).

Application Databases (OLTP)

- OLTP systems are often referred to as transactional databases, but this does not necessarily imply that the system in question supports atomic transactions.
- More generally, OLTP databases support low latency and high concurrency.
(atomicity, consistency, isolation, durability).

Online Analytical Processing System

- Online analytical processing (OLAP) system is built to run large analytics queries and is typically inefficient at handling lookups
- Of individual records. For example, modern column databases are optimized to scan large volumes of data, dispensing with indexes to improve scalability and scan performance.
- Why are we talking about them in chapter on source systems? E.g, a data warehouse might serve data used to train an ML model. Or, an OLAP system might serve a reverse ETL workflow.

Logs

- A log captures information about events that occur in systems.
 - e.g, a log may capture traffic and usage patterns on a web server.
 - Your desktop computer's operating system (Windows, macOS, Linux) logs events as the system boots and when applications start or crash.
- Logs are a rich data source, potentially valuable for downstream data analysis, ML, and automation.

Database logs

- Database logs are essential enough that they deserve more detailed coverage.
- Write ahead logs (typically, binary files stored in a specific database-native format) play a crucial role in database guarantees and recoverability. The database server receives write and update requests to a database table, storing each operation in the log before acknowledging the request.
- The acknowledgment comes with a log-associated guarantee: even if the server fails, it can recover its state on reboot by completing the unfinished work from the logs.
- Database logs are extremely useful in data engineering, especially for CDC to generate event streams from database changes.

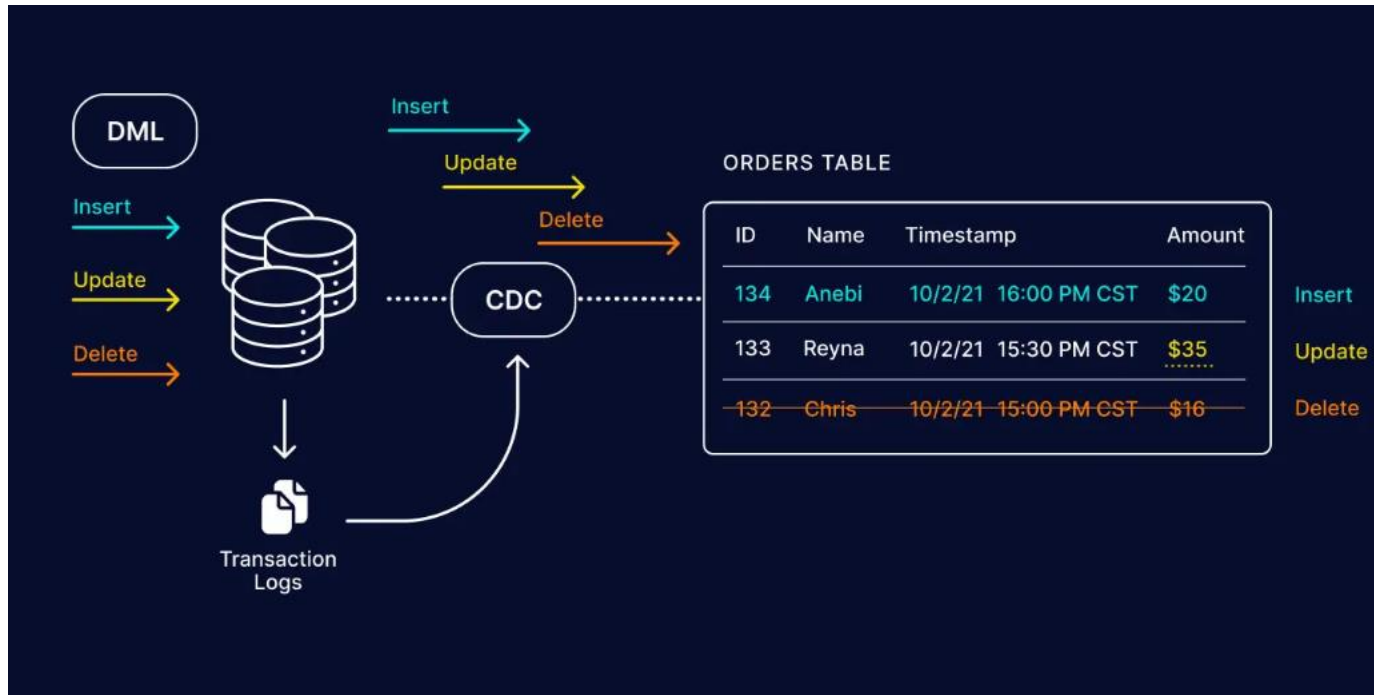
CRUD

- CRUD (create, read, update, and delete) is a transactional pattern commonly used in programming and represents the four basic operations of persistent storage.
 - CRUD is the most common pattern for storing application state in a database.
- CRUD is a widely used pattern in software applications, and you'll commonly find CRUD used in APIs and databases.
- For example, a web application will make heavy use of CRUD for RESTful HTTP requests and storing and retrieving data from a database.

Change Data Capture

- Change data capture (CDC) is a method for extracting each change event (insert, update, delete) that occurs in a database.
- CDC is frequently leveraged to replicate between databases in near real time or create an event stream for downstream processing.

Change Data Capture



Src: <https://www.striim.com/blog/change-data-capture-cdc-what-it-is-and-how-it-works/>

Change Data Capture

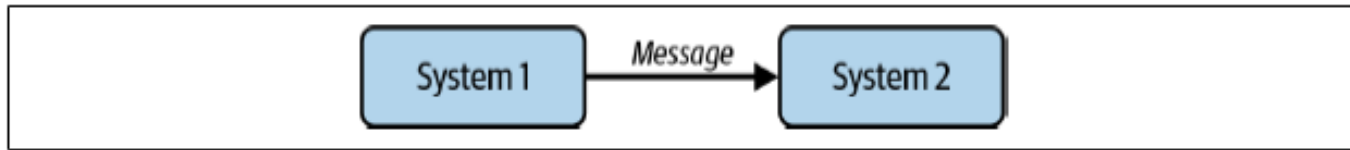
- It is common to need to know the changes within a table.
 - E.g: relational systems often implement CDC to help track changes and minimize effort in synchronizing systems.
- Similarly, Delta tables have CDF (Change Data Feed). CDF allows row-level tracking. CDF is critical to Lakehouse architecture in that it is the enabling technology of incremental processing. With CDF, we can identify the subset of records to process from layer to layer.
- This reduces processing time and, in turn, costs.

Src: <https://www.striim.com/blog/change-data-capture-cdc-what-it-is-and-how-it-works/>

Messages & Streams

- Related to event-driven architecture, two terms that you'll often see used interchangeably are message queue and streaming platform, but a subtle but essential difference exists between the two.
- Defining and contrasting these terms is worthwhile since they encompass many big ideas related to source systems and practices and technologies spanning the entire data engineering lifecycle.

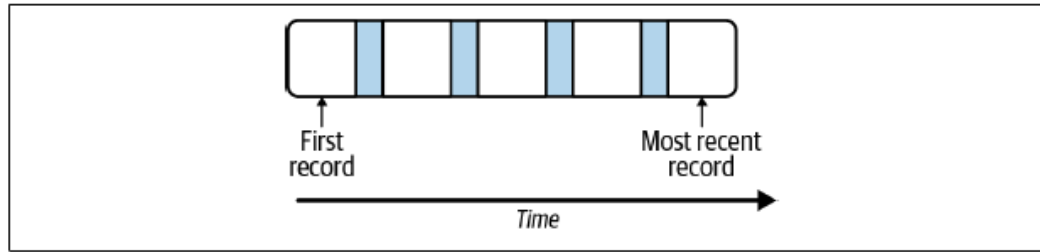
Messages & Streams



- A message is raw data communicated across two or more systems. For example, we have System 1 and System 2, where System 1 sends a message to System 2.
- These systems could be different microservices, a server sending a message to a serverless function, etc.
- A message is typically sent through a message queue from a publisher to a consumer, and once the message is delivered, it is removed from the queue

Src: Fundamentals of Data Engineering (Reis & Housley)

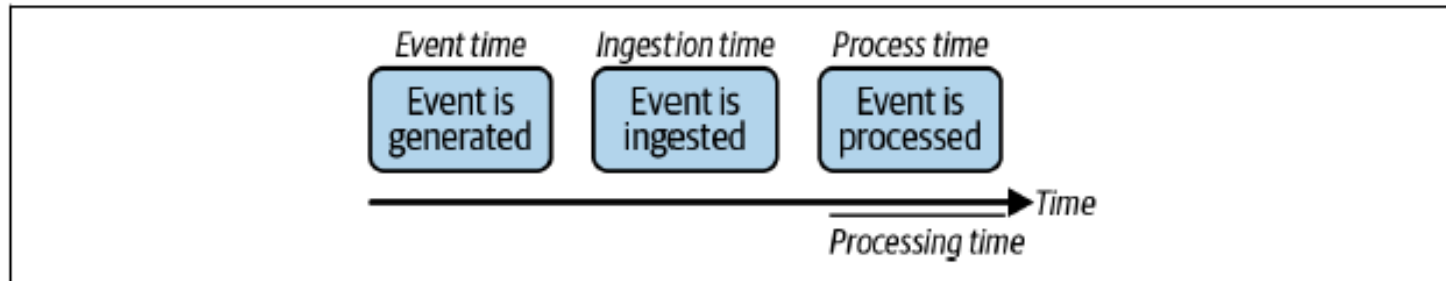
Messages & Streams



- Stream: An append-only log of event records.
- Storage: Events are stored in order (by timestamp or ID) on event-streaming platforms.
- Use Case: Ideal for tracking what happened over many events.
- Retention: Records persist for weeks or months. It supports:
 - Aggregations across records.
 - Rewinding to past events.

Src: Fundamentals of Data Engineering (Reis & Housley)

Messages & Streams – Types of Time.



- While time is an essential consideration for all data ingestion, it becomes that much more critical and subtle in the context of streaming, where we view data as continuous and expect to consume it shortly after it is produced.
- The key types of time when ingesting data: the time that the event is generated, when it's ingested and processed, and how long processing took.

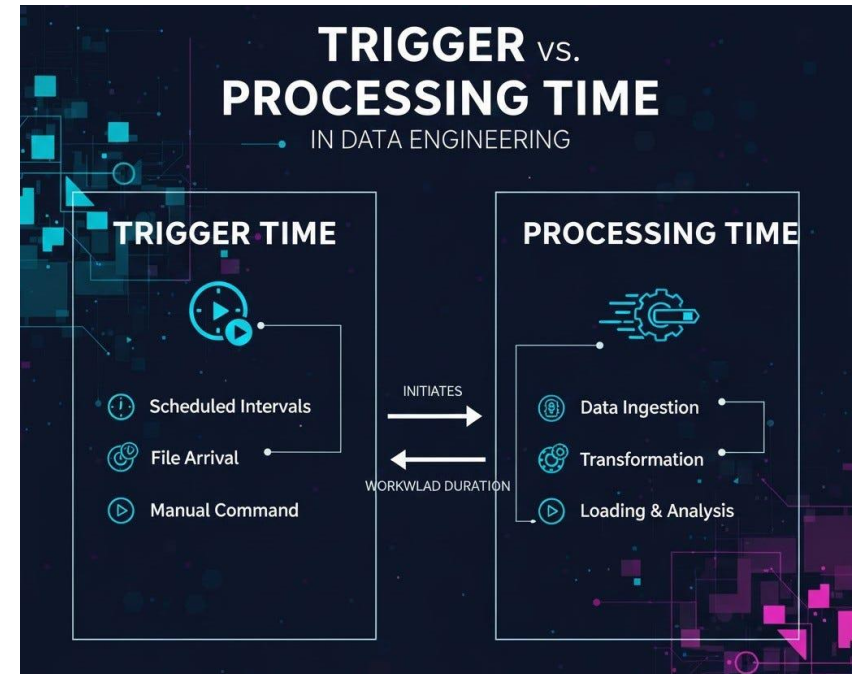
Messages & Streams – Types of Time.

- **Event time indicates:** when an event is generated in a source system, including the timestamp of the original event itself.
- An undetermined time lag will occur upon event creation, before the event is ingested and processed downstream. Always include timestamps for each phase through which an event travels.
- Log events as they occur and at each stage of time—when they're created, ingested, and processed. Use these timestamp logs to accurately track the movement of your data through your data pipelines.

Messages & Streams – Types of Time.

- **Ingestion time indicates** when an event is ingested from source systems into a message queue, cache, memory, object storage, a database, or any place else that data is stored. After ingestion, data may be processed immediately; or within minutes, hours, or days; or simply persist in storage indefinitely.
- **Process time occurs** after ingestion time, when the data is processed (typically, a transformation).

Processing time is how long the data took to process, measured in seconds, minutes, hours, etc.



[Source](#)

Source Systems & Security

Security is critical, and the last thing you want is to accidentally create a point of vulnerability in a source system. Here are some areas to consider:

- Is the source system architected so data is secure and encrypted, both with data at rest and while data is transmitted?
- Do you have to access the source system over the public internet, or are you using a virtual private network (VPN)?

Source Systems & Security

- Keep passwords, tokens, and credentials to the source system securely locked away.

For example, if you're using Secure Shell (SSH) keys, use a key manager to protect your keys; the same rule applies to passwords—use a password manager or a single sign-on (SSO) provider.

- Do you trust the source system? Always be sure to trust but verify that the source system is legitimate. You don't want to be on the receiving end of data from a malicious actor.

Source Systems & Data Management

Data management of source systems is challenging for data engineers. In most cases, you will have only **peripheral control**—if any control at all—over source systems and the data they produce.

To the extent possible, you should understand the way data is managed in source systems since this will directly influence how you ingest, store, and transform the data.

Here are some areas to consider:

Data governance

- Are upstream data and systems governed in a reliable, easy-to-understand fashion?
- Who manages the data?

Src: Fundamentals of Data Engineering (Reis & Housley)

Source Systems & Data Management

Data quality

- How do you ensure data quality and integrity in upstream systems? Work with source system teams to set expectations on data and communication.

Schema

- Expect that upstream schemas will change. Where possible, collaborate with source system teams to be notified of looming schema changes.

Master data management

- Is the creation of upstream records controlled by a master data management practice or system?

Source Systems & Data Management

Privacy and ethics

- Do you have access to raw data, or will the data be obfuscated? What are the implications of the source data? How long is it retained? Does it shift locations based on retention policies?

Regulatory

- Based upon regulations, are you supposed to access the data?

Source Systems & Data Ops

Operational Excellence:

- Applies to DevOps, DataOps, MLOps, and XOps across the entire stack.
- Supports the data engineering process and lifecycle, though not always fully realized.

Stakeholder Considerations:

- Work with stakeholders who control source systems and their data.
- Ensure you can observe and monitor system uptime and usage.
- Be ready to respond to incidents promptly.

Source Systems & Data Architecture

Reliability: You should also understand how the upstream architecture is designed and its strengths and weaknesses. Talk often with the teams responsible for the source systems to understand the factors discussed in this section and ensure that their systems can meet your expectations.

Reliability, Durability, Availability, & People

Source Systems & Orchestration?

When orchestrating within your data engineering workflow, you'll primarily be concerned with making sure your orchestration can access the source system, which requires the correct network access, authentication, and authorization.

Here are some things to think about concerning orchestration for source systems:

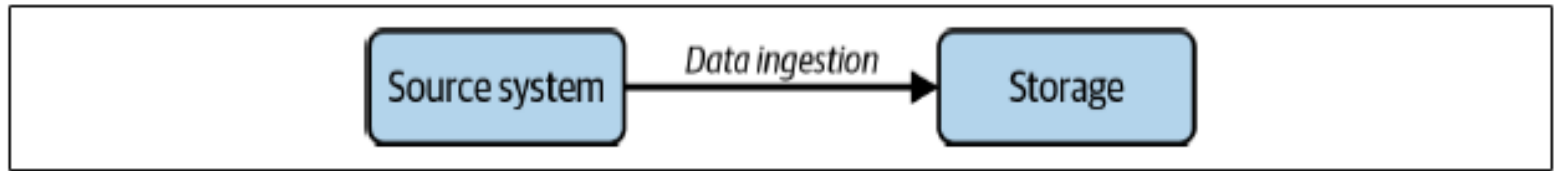
Cadence and frequency: Is the data available on a fixed schedule, or can you access new data whenever you want?

Common frameworks

- Do the software and data engineers use the same container manager, such as Kubernetes?
- Would it make sense to integrate application and data workloads into the same Kubernetes cluster?

Src: Fundamentals of Data Engineering (Reis & Housley)

Data Ingestion



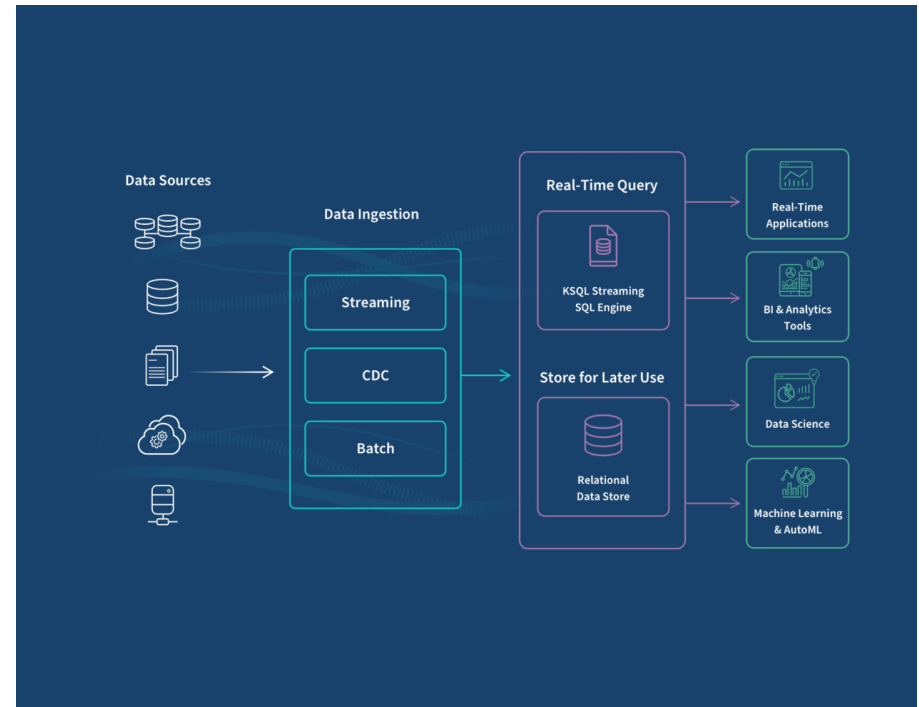
- Data ingestion is the process of moving data from one place to another.
- Data ingestion implies data movement from source systems into storage in the data engineering lifecycle, with ingestion as an intermediate step.

Key Considerations During Ingestion Phase

- What's the use case for the data I'm ingesting?
- Can I reuse this data and avoid ingesting multiple versions of the same dataset?
- Where is the data going? What's the destination?
- How often should the data be updated from the source?
- What is the expected data volume?
- What format is the data in? Can downstream storage and transformation accept this format?
- Is the source data in good shape for immediate downstream use? That is, is the data of good quality? What post-processing is required to serve it? What are data-quality risks (e.g., could bot traffic to a website contaminate the data)?
- Does the data require in-flight processing for downstream ingestion if the data is from a streaming source?

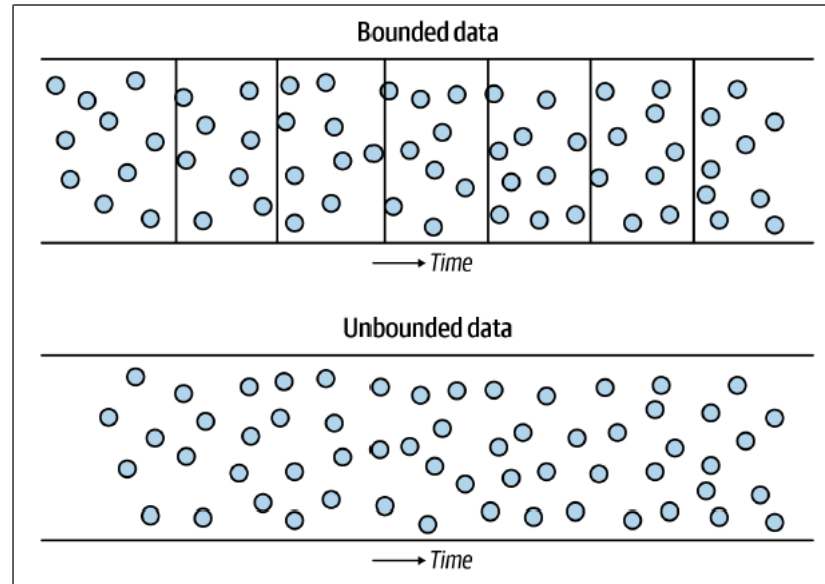
Key Factors impacting Ingestion architectures:

- Bounded versus unbounded
- Frequency
- Synchronous versus asynchronous
- Push vs. Pull Ingestion
- Serialization and deserialization
- Throughput and scalability
- Reliability and durability
- Payload
- Push versus pull versus poll patterns



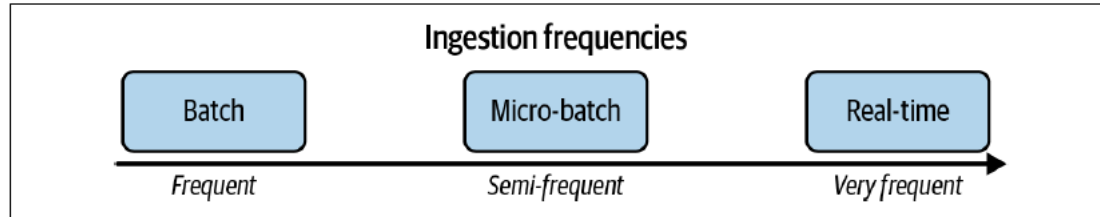
[Source](#)

Bounded vs. UnBounded



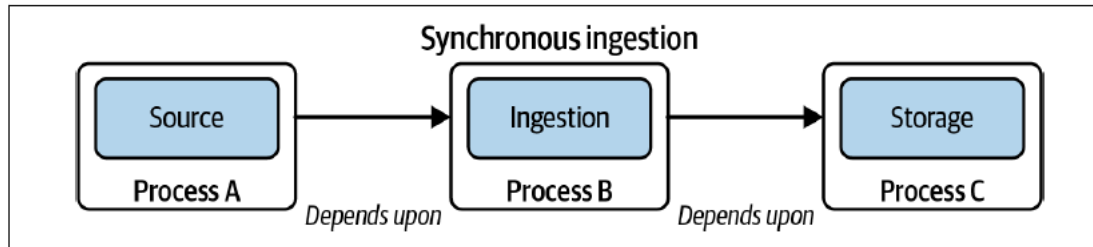
- Unbounded data is data as it exists in reality, as events happen, either sporadically or continuously, ongoing and flowing.
- Bounded data is a convenient way of bucketing data across some sort of boundary, such as time. mantra: All data is unbounded until it's bounded.

Frequency

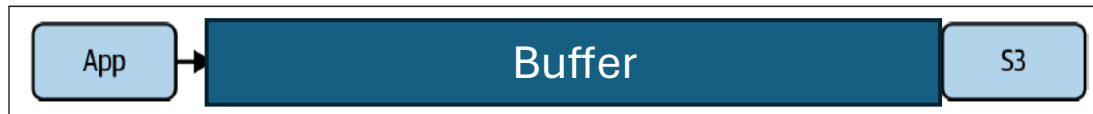


- Ingestion processes can be batch, micro-batch, or real-time.
- real-time and streaming interchangeably

Synchronous vs. Asynchronous



- complex dependencies and are tightly coupled
- If process A fails, processes B and C cannot start; if process B fails, process C doesn't start.

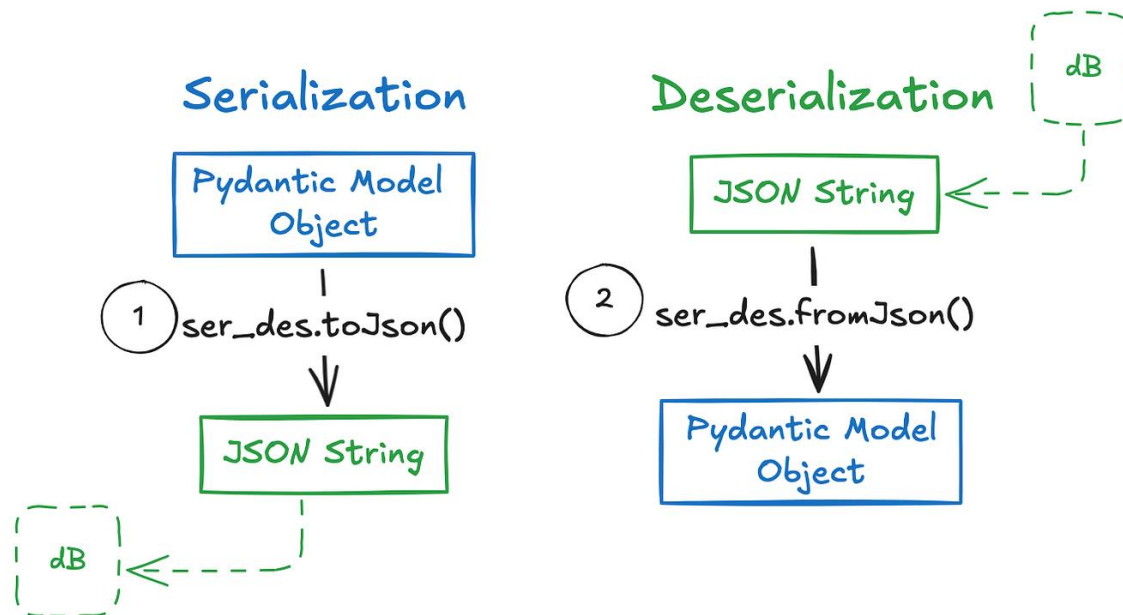


- the asynchronous pipeline can process data items as they become available in parallel across the a cluster

Src: Fundamentals of Data Engineering (Reis & Housley)

Serialization/Deserialization

- Moving data from source to destination involves serialization and deserialization. As a reminder, serialization means encoding the data from a source and preparing data structures for transmission and intermediate storage stages.
- When ingesting data, ensure that your destination can deserialize the data it receives.



Src: Fundamentals of Data

Throughput and Scalability

- Data throughput and system scalability become critical as your data volumes grow and requirements change. Design your systems to scale and shrink to flexibly match the desired data throughput
- Another thing to consider is your ability to handle bursty data ingestion.
- Whenever possible, use managed services that handle the throughput scaling for you.

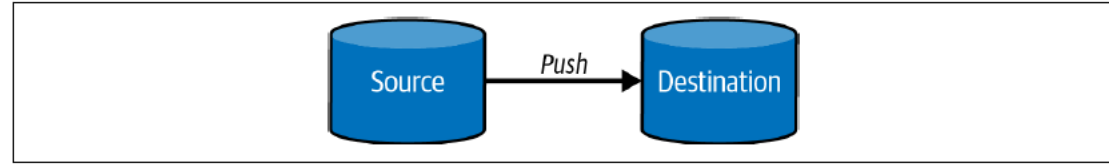
Reliability, Durability, Scalability, and Throughput

<u>Concept</u>	<u>What It Means</u>	<u>Why It Matters</u>	<u>Example</u>
Data Throughput	The amount of data a system can process in a given time	As data volume grows, systems must handle higher ingestion and processing rates	System processes 10,000 events per second and scales to 100,000 during peak hours
Scalability	Ability of a system to increase or decrease resources based on demand	Prevents over-provisioning and avoids system overload	Auto-scaling compute nodes in a cloud environment during traffic spikes
Bursty Ingestion	Handling sudden spikes in incoming data	Prevents data loss or backlogs during peak loads	Handling a 5× traffic spike during a product launch
Managed Services	Cloud services that automatically handle scaling and infrastructure	Reduces operational complexity and scaling risk	Using AWS Kinesis or Databricks instead of self-managing servers
Reliability	High uptime and proper failover mechanisms	Ensures continuous ingestion without interruptions	Automatic failover to a secondary region if a primary server fails
Durability	Guarantee that data is not lost or corrupted	Protects against permanent data loss	Data replicated across multiple storage nodes or availability zones

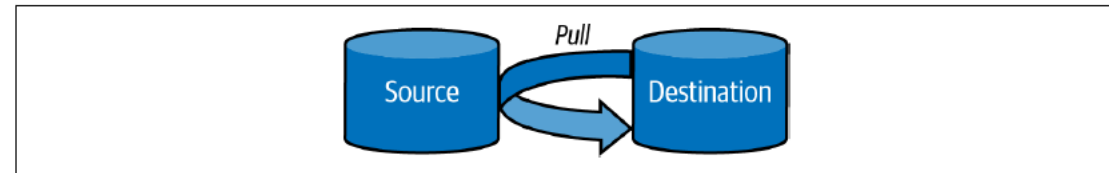
Src: Fundamentals of Data Engineering (Reis & Housley)

Push vs. Pull vs. Poll

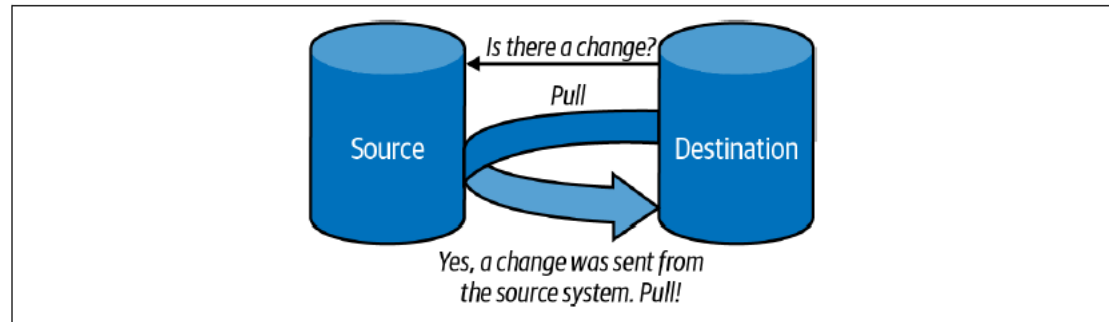
A push strategy involves a source system sending data to a target



while a pull strategy entails a target reading data directly from a source.



Polling involves periodically checking a data source for any changes. When changes are detected, the destination pulls the data as it would in a regular pull situation.



ETL vs. ELT

ETL (Extract, Transform, Load)

- Extract data from source systems
- Transform data before loading
- Load cleaned data into warehouse
- Common in traditional/on-prem systems

*ETL transforms *before* loading.

*ELT transforms *after* loading.

ELT (Extract, Load, Transform)

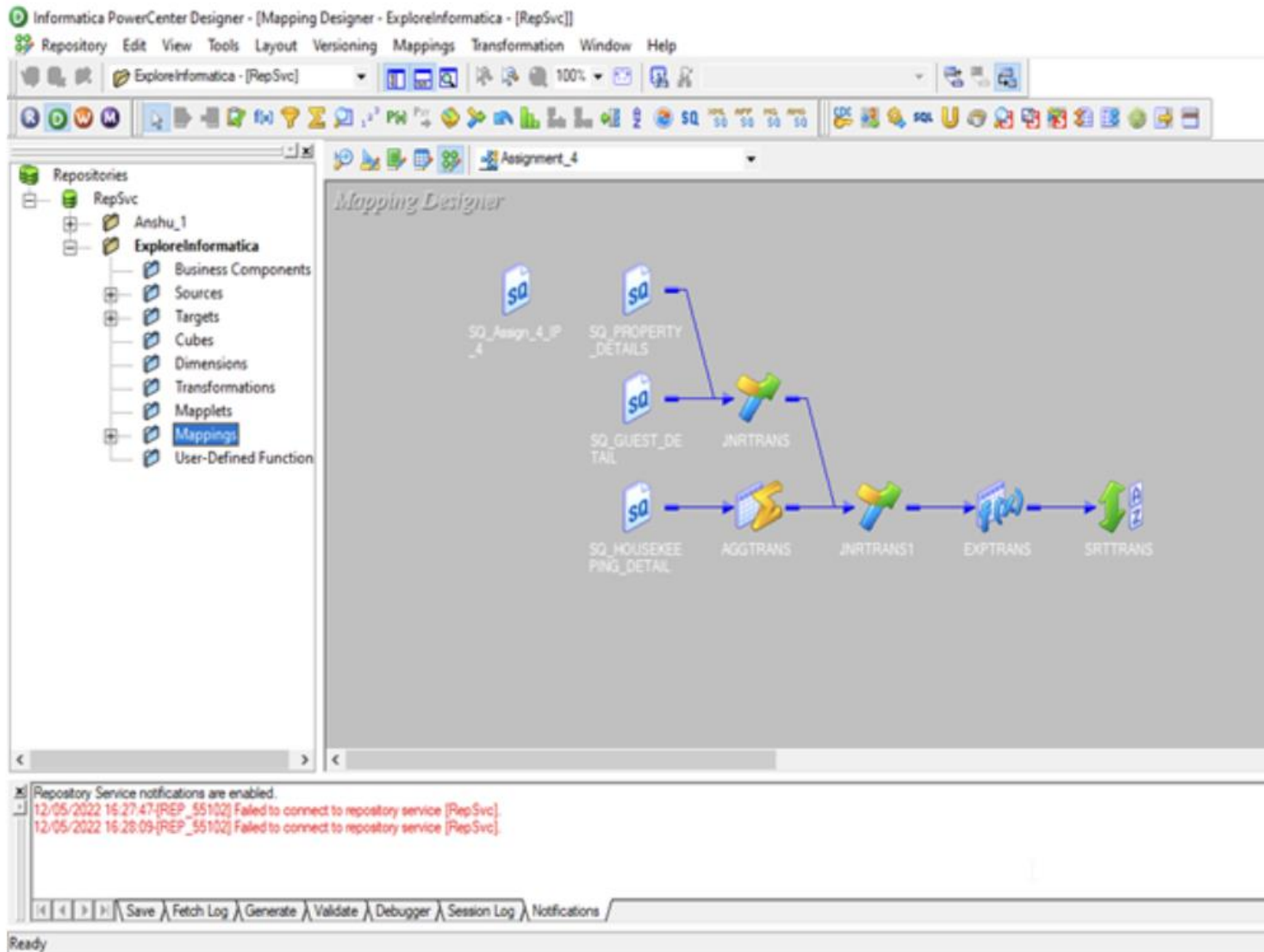
- Extract data from source systems
- Load raw data into warehouse first
- Transform inside the warehouse
- Common in modern cloud platforms

Modern cloud data engineering typically favors ELT.

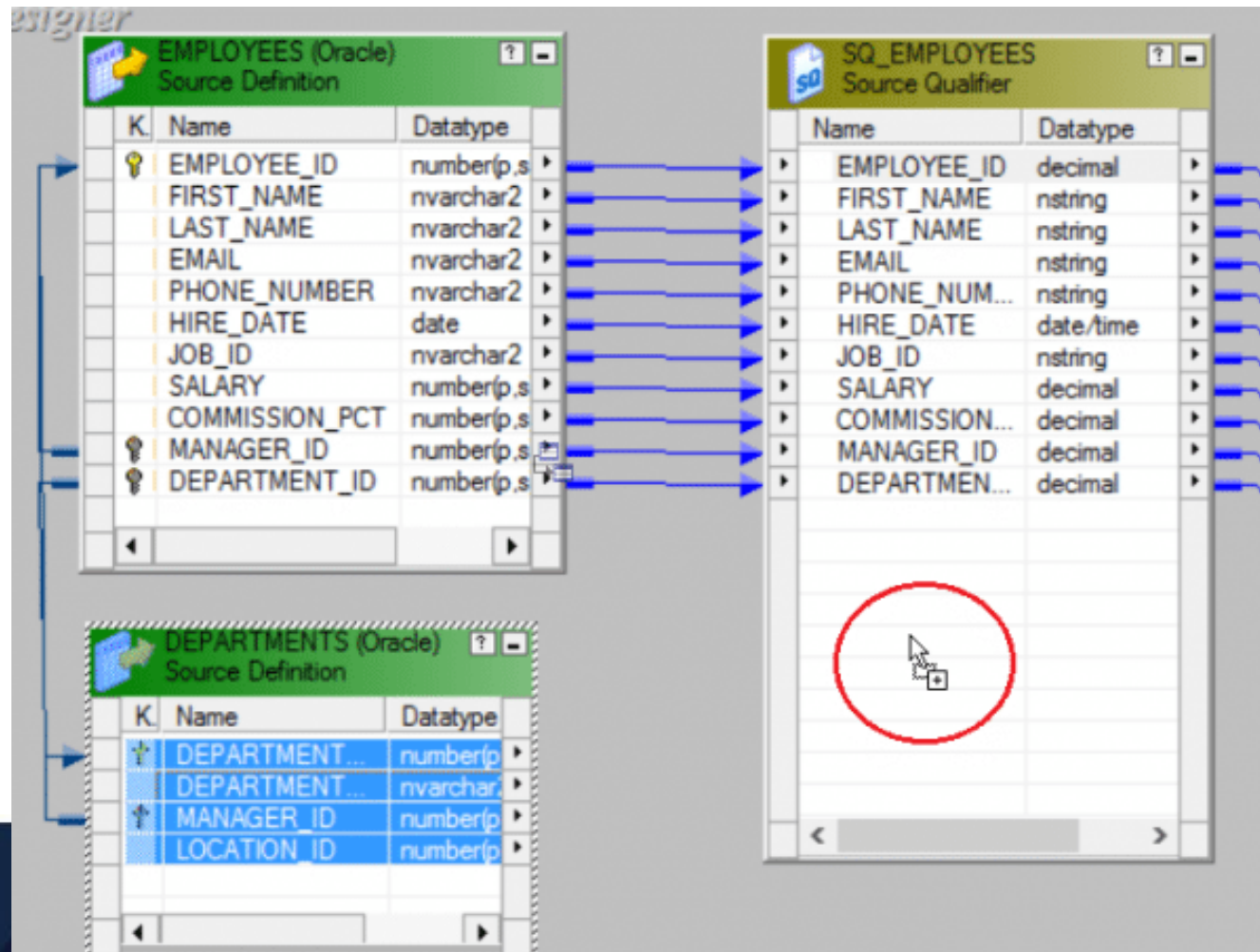
Common ETL/ELT Tools & Platforms

<u>Category</u>	<u>Tool</u>	<u>Primary Purpose</u>	<u>Typical Use Case</u>
Data Integration	Fivetran	Managed data extraction & loading	Sync SaaS data into warehouse
Data Integration	Stitch	Cloud ETL service	Lightweight pipeline setup
Data Integration	AWS Glue	Serverless ETL service	Cloud-native data pipelines
Enterprise ETL	Informatica	Enterprise-grade ETL platform	Large enterprise environments
Enterprise ETL	Talend	Data integration & transformation	Hybrid cloud + on-prem systems
Transformation	dbt	SQL-based transformations	ELT workflows in warehouse
Distributed Processing	Apache Spark	Large-scale distributed transformation	Big data processing
Lakehouse Platform	Databricks	Unified data engineering & analytics platform built on Spark	ELT, ML, streaming, lakehouse architecture
Orchestration	Apache Airflow	Workflow scheduling & monitoring	Pipeline automation
Orchestration	Prefect	Modern workflow orchestration	Python-native workflows
Streaming	Apache Kafka	Real-time event streaming	Streaming ingestion
Streaming	Spark Streaming	Real-time distributed processing	Continuous transformations
Cloud Warehouse	Snowflake	Cloud data warehouse	ELT processing
Cloud Warehouse	BigQuery	Serverless data warehouse	SQL-based transformations
Cloud Warehouse	Amazon Redshift	Managed data warehouse	Structured analytics

Informatica Example



Informatica Example



What is a Lakehouse?

Lakehouse Design

- All-important building efforts begin with design. Before building a house, we develop blueprints that tell the builders how to build the house. Similarly, when building a Data lakehouse, we start with a design.
- A lakehouse is a data platform architecture paradigm that combines the best features of data lakes and data warehouses.
- A modern lakehouse is a highly scalable and performant data platform hosting both raw and prepared data sets for quick business consumption and to drive advanced business insights and decisions. It breaks data silos and allows seamless, secure data access to authorized users across the enterprise on one platform.

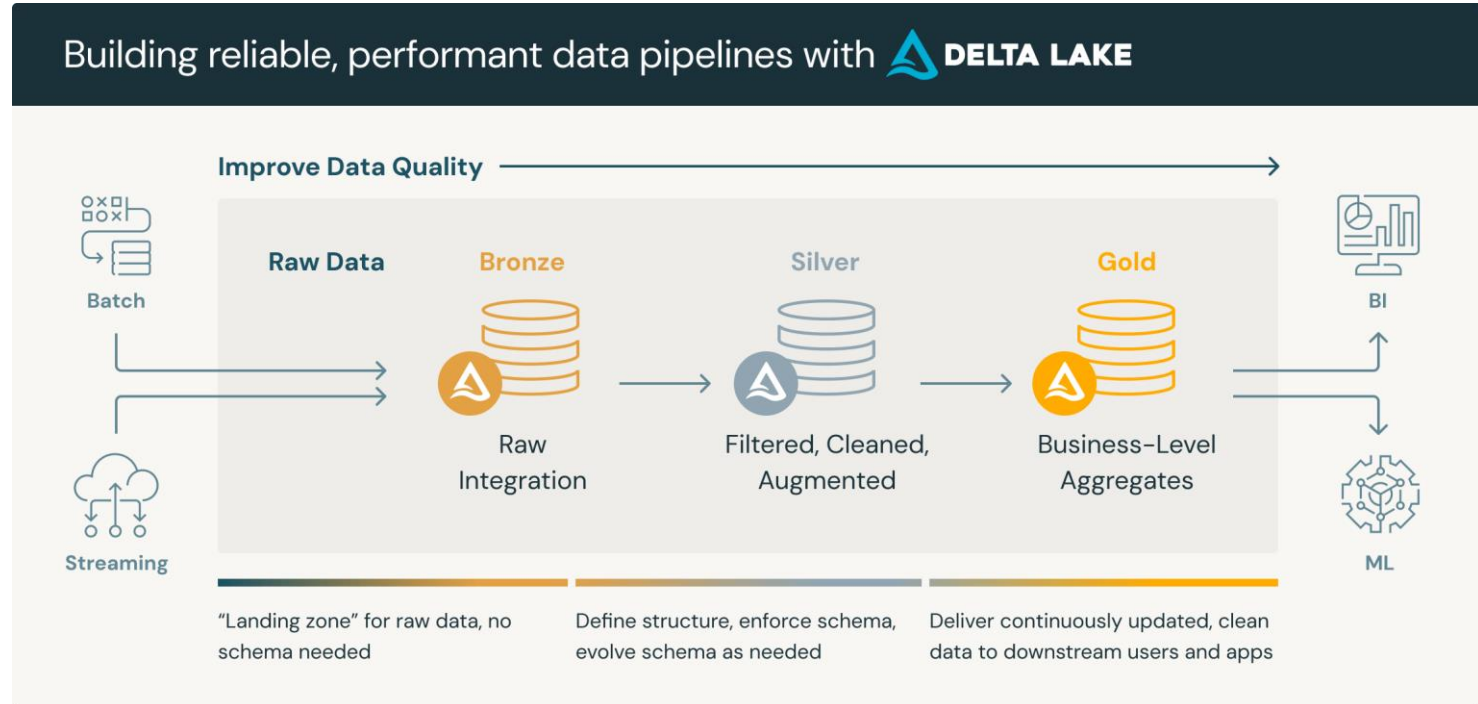
Lakehouse Design

Benefits of a lakehouse architecture

- Simple data model
- Easy to understand and implement
- Enables incremental ETL
- Can recreate your tables from raw data at any time
- ACID transactions, time travel
- Combines the best of both worlds (Data lake + Warehouse). Please review slides in

Lecture 2

Medallion Architecture



3 Key layers

Src: <https://www.databricks.com/glossary/medallion-architecture>

Bronze Layer

- The Bronze layer (raw data) is where we land all the data from external source systems. The table structures in this layer correspond to the source system table structures "as-is," along with any additional metadata columns that capture the load date/time, process ID, etc.
- The focus in this layer is quick Change Data Capture and the ability to provide an historical archive of source (cold storage), data lineage, auditability, reprocessing if needed without rereading the data from the source system.

Silver layer (cleansed and conformed data)

- In the Silver layer of the lakehouse, the data from the Bronze layer is matched, merged, conformed and cleansed ("just-enough") so that the Silver layer can provide an "Enterprise view" of all its key business entities, concepts and transactions. (e.g. master customers, stores, non-duplicated transactions and cross-reference tables).
- The Silver layer brings the data from different sources into an Enterprise view and enables self-service analytics for ad-hoc reporting, advanced analytics and ML. It serves as a source for Departmental Analysts, Data Engineers and Data Scientists to further create projects and analysis to answer business problems via enterprise and departmental data projects in the Gold Layer.

Src: <https://www.databricks.com/glossary/medallion-architecture>

Silver layer (cleansed and conformed data)

- In the lakehouse data engineering paradigm, typically the ELT methodology is followed vs. ETL - which means only minimal or "just-enough" transformations and data cleansing rules are applied while loading the Silver layer.
- Speed and agility to ingest and deliver the data in the data lake is prioritized, and a lot of project-specific complex transformations and business rules are applied while loading the data from the Silver to Gold layer. From a data modeling perspective, the Silver Layer has more 3rd-Normal Form like data models

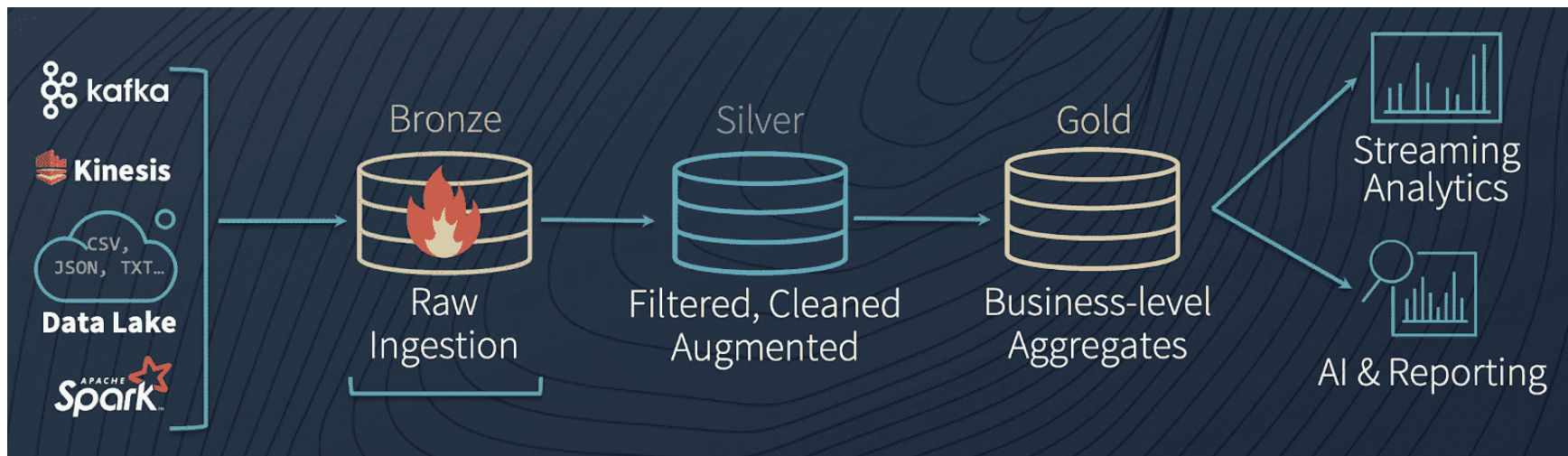
Gold layer (curated business-level tables)

- Data in the Gold layer of the lakehouse is typically organized in consumption-ready "project-specific" databases. The Gold layer is for reporting and uses more de-normalized and read-optimized data models with fewer joins. The final layer of data transformations and data quality rules are applied here.
- Final presentation layer of projects such as Customer Analytics, Product Quality Analytics, Inventory Analytics, Customer Segmentation, Product Recommendations, Marketing/Sales Analytics etc. fit in this layer. We see a lot of Kimball style star schema-based data models or Inmon style Data marts fit in this Gold Layer of the lakehouse.

Src: <https://www.databricks.com/glossary/medallion-architecture>

To Raw or Not to Raw

One area that is not always clear is if the raw layer should be used. It can be argued that the contents of the raw layer are identical to those of the bronze layer. An approach is to include items coming from file-based systems in the raw zone in their native format. Data from relational or other enterprise-class systems typically skip the raw layer and land directly into bronze.



<https://www.databricks.com/blog/2019/08/14/productionizing-machine-learning-with-delta-lake.html>

Src: Databricks Lakehouse Platform Cookbook Dr. Alan Dennis

What is DeltaLake?

- Delta Lake is an open-source storage layer that adds reliability and performance features to data lakes.
- It was originally created by Databricks and is now part of the Linux Foundation.
- Delta Lake is a core technology in the Data Lakehouse architecture.

Is It a Data Lake or Lakehouse?

- A traditional data lake stores raw files (e.g., Parquet, CSV) but lacks transaction guarantees.
- A lakehouse combines data lake storage with data warehouse reliability.
- Delta Lake enables the lakehouse model by adding database-like features on top of data lake storage.

When is a “raw” layer necessary

- **Unstructured or Semi-Structured Sources:** e.g. Logs, JSON APIs, IoT data, or flat files (CSV, Avro). Raw storage is needed to capture the data in its original, unaltered form for compliance, auditability, or reprocessing.
- **Volatile or Streaming Sources:** e.g. Kafka streams or real-time event hubs. The raw layer serves as a "catch-all" for every message, ensuring no data is lost even if downstream processing fails.
- **Compliance or Traceability:** Industries like healthcare, finance, or government often require a full historical record of the original source data. The raw layer acts as a legal or audit trail.
- **Source System Does Not Maintain History:** APIs or databases without CDC or historical logs. A raw layer ensures you have an immutable backup of the incoming data for lineage or replay purposes..

Lakehouse Tracker

Source		Raw	Bronze		Silver			Gold		
Name	Table Folder	Folder	Folder	Table Name	Type	Folder	Table	Type	Folder	Table

this is not an exhaustive list of items to track but a starting point. For example, you likely will want to denote the type of ingestion, such as full or incremental. Additionally, the data update rate within the data items is important when designing ingestion processes.

Rules of Movements within Layers

Each team member must understand what changes can be performed when crossing into a given layer. This must be documented early in a project to ensure consistency. This need not be a lengthy document; it could be contained in a PowerPoint presentation

Source to Raw

- Files Housed in a File-based System (FTP, ADLS, etc.)
 - Land in Raw in a folder based on the source system's name relative to the raw folder's root
 - Do not change the format
 - If incrementally ingesting, include a time-sliced folder structure
 - If the source is not trustworthy, include full loads in a time-sliced folder structure and track the most recently ingested
- Data is Housed in Enterprise-class Systems (such as Relational)
 - Consider not using raw. If raw is required
 - land files in Parquet format
 - If incrementally ingesting, include a time-sliced folder structure

Source to Bronze

- Avoid changing names of data items other than
 - To indicate their source system
 - Make them unique
 - Indicate the item is a bronze item
- Avoid changing the shape of the data
- Changing data types is permissible to make them compatible with Delta data types
- Adding a column for tracking purposes is permissible, but it should be clear the column did not come from the source system (e.g. `append_systemadded`)

Raw to Bronze

- Apply Delta schema to the source files
- Do not remove columns
- Do not expand nested columns to multiple columns
- As with Source to Bronze, adding tracking columns is permissible
- Convert to the most appropriate data type for all columns
- Avoid leaving columns as strings
- Avoid deleting bronze tables, instead replace them so that history is preserved
- Generally, append to the contents of the bronze table

Bronze to Silver

- Do not remove columns
- Adding or updating tracking columns is acceptable
- Data type conversions should not be necessary as they were done entering the bronze layer
- Remove duplications, selecting the most recent version based on identifier and modification date
- Merge into silver, updating existing and inserting new items
- Optionally remove items that have been deleted, preferring a soft delete strategy if possible
- Apply constraints and expectations to ensure quality data

Silver to Silver

- Silver tables are fit for use but are not created for specific use cases like gold tables.
- Transformations are often needed to convert a silver table into a structure suitable for constructing a gold table.
 - Intermediate views
 - Tables (for performance reasons)

Same-Layer Transformations/Combining Tables:

- Data enrichment
- Denormalization

Expectations and Constraints:

- Ensure transformations do not introduce invalid information in silver tables.
- Maintain record uniqueness if columns are removed.

Data Shape Changes:

- The shape of the data will change significantly during these operations.
- New tables are built for future needs and often involve joining tables together.

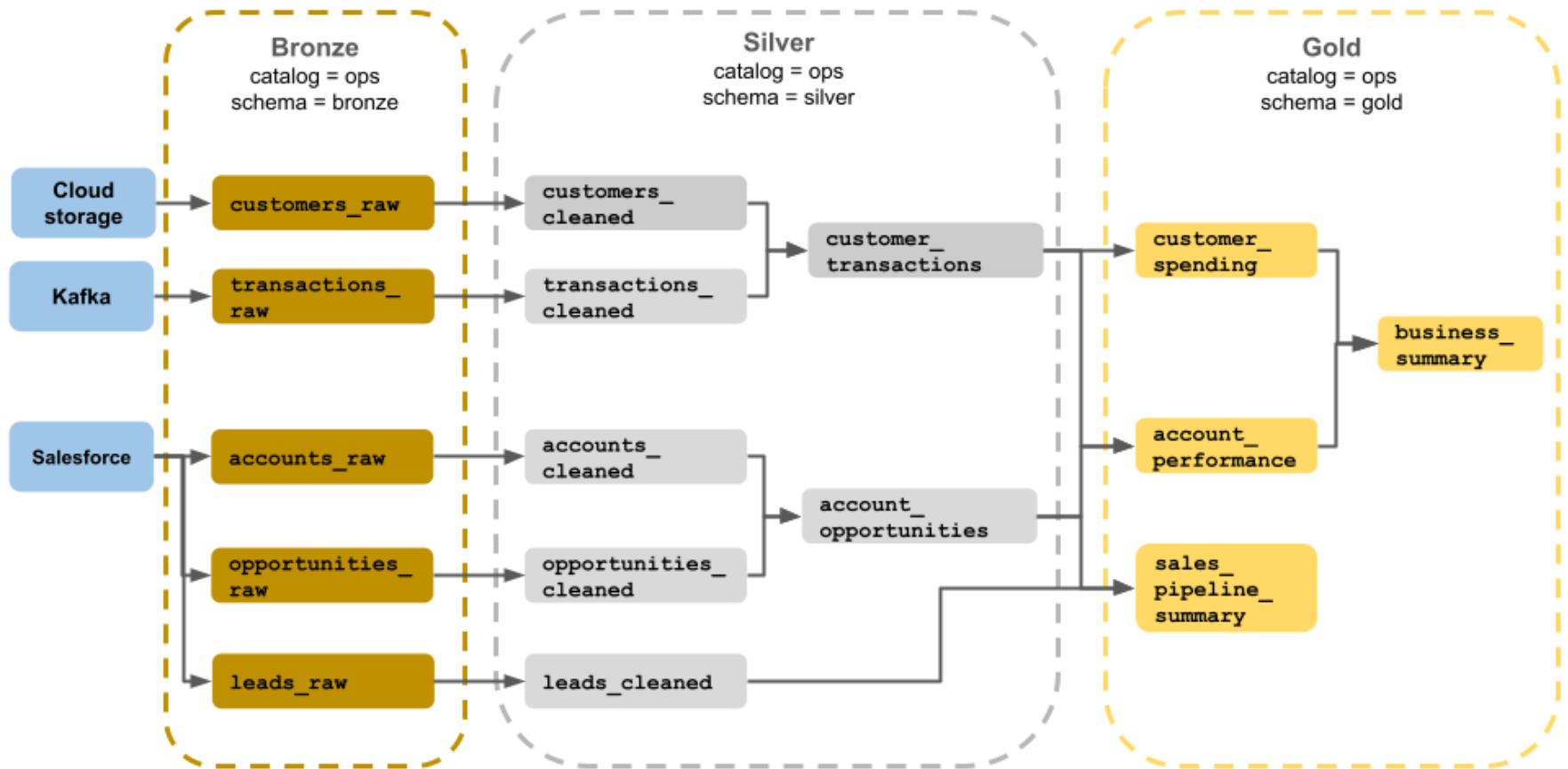
Pruning Unneeded Columns:

- Not all columns are necessary after joining tables.
- Create a set of CORE tables with most valuable columns.

Silver to Gold and Gold to Gold

- Created for a specific business need
- Use constraints and expectations to ensure quality data
- Table shape is driven by business requirements being satisfied

End To End View



Src: https://learn.microsoft.com/en-us/azure/databricks/_static/images/lakehouse-architecture/medallion-architecture.png

Data Profiling

- Data profiling, or data archeology, is the process of reviewing and cleansing data to better understand how it's structured and maintain data quality standards within an organization.
- The main purpose is to gain insight into the quality of the data by using methods to review and summarize it, and then evaluating its condition. The work is typically performed by data engineers who will use a range of business rules and analytical algorithms.

Src: <https://www.ibm.com/topics/data-profiling>

Data Profiling

- Data profiling evaluates data based on factors such as accuracy, consistency, and timeliness to show if the data is lacking consistency or accuracy or has null values. A result could be something as simple as statistics, such as numbers or values in the form of a column, depending on the data set. Data profiling can be used for projects that involve data warehousing or business intelligence and is even more beneficial for big data. Data profiling can be an important precursor to data processing and data analytics.
- Data profiling plays a crucial role in understanding and ensuring the quality of data as it moves through the various stages of the lakehouse design layers.

Src: <https://www.ibm.com/topics/data-profiling>

Data Profiling

- particularly important during the data generation and ingestion phases for several reasons:
- Understanding Source Data: Data profiling helps in comprehending the structure, content, and quality of the data being generated or ingested.
- Data Quality Assessment: It allows for early detection of data quality issues, which is crucial before data enters the pipeline.
- Informing Ingestion Strategies: Profiling can help determine the most appropriate ingestion methods (batch vs. streaming) based on the data characteristics.
- Optimizing Storage and Processing: Understanding data patterns through profiling can inform decisions on storage solutions and processing requirements.

Src: <https://panoply.io/analytics-stack-guide/data-profiling-best-practices/>

Types of Data Profiling

- Structure discovery (validating data formatting and consistency)
- Content discovery (checking data quality and standardization)
- Relationship discovery (uncovering connections between datasets)

Src: <https://panoply.io/analytics-stack-guide/data-profiling-best-practices/>

Data Profiling Techniques

- Column Profiling
 - Analyzes individual columns for data types, patterns, and basic statistics
 - Example: Identifying the most common values in a "City" column
- Cross-Column Profiling
 - Examines relationships between different columns in the same table
 - Example: Discovering correlations between "Age" and "Income" columns
- Cross-Table Profiling
 - Analyzes relationships between tables
 - Example: Identifying foreign key relationships between "Orders" and "Customers" tables

Src: <https://www.netsuite.com/portal/resource/articles/data-warehouse/data-profiling.shtml>

Data Profiling in Databricks

L4_P Python ▾

File Edit View Run Help [Last edit was now](#)

▶ Run all

● Walid Hassan's Cluster ▾

Share



✓ 2 minutes ago (3s)

1

SQL



%sql

```
select * from samples.nyctaxi.trips
```

▶ (2) Spark Jobs

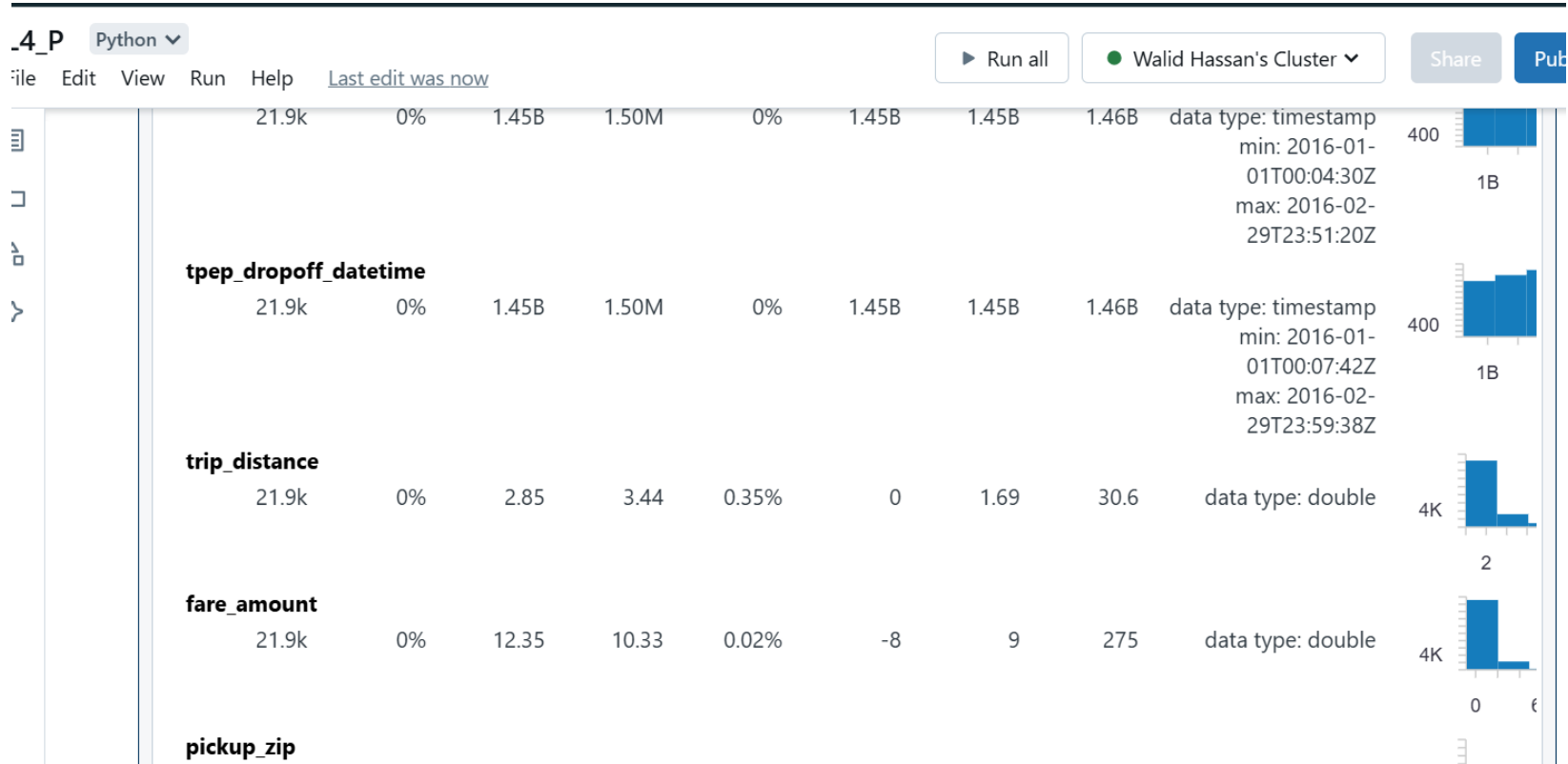
▶ _sqldf: pyspark.sql.dataframe.DataFrame = [tpep_pickup_datetime: timestamp, tpep_dropoff_datetime: timestamp ... 4 more fields]

Table ▾

Visualization
Data Profile

	tpep_pickup_datetime	tpep_dropoff_datetime	1.2 trip_distance	1.2 fare_amount	1 ² ₃ pickup_zip
3	2016-02-29T22:25:33.000+00:00	2016-02-08T07:44:14.000+00:00	0.9	5.5	10119
4	2016-02-29T22:25:33.000+00:00	2016-02-29T22:38:09.000+00:00	3.5	13.5	10001
5	2016-02-03T17:21:02.000+00:00	2016-02-03T17:23:24.000+00:00	0.3	3.5	10028
6	2016-02-10T00:47:44.000+00:00	2016-02-10T00:53:04.000+00:00	0	5	10038
7	2016-02-19T03:24:25.000+00:00	2016-02-19T03:44:56.000+00:00	6.57	21.5	10001

Data Profiling in Databricks



count, missing, mean, std dev, zeros, min, median, max

Data Profiling

L4_P Python ▾

File Edit View Run Help [Last edit was 12 minutes ago](#)

▶ Run all

● Walid Hassan's Cluster ▾

Share

Pub

▶ ✓ 12:32 AM (5s)

3

```
dbutils.data.summarize(df)
```

▶ (5) Spark Jobs

profiles generated in approximate mode ⓘ

Sort by

Feature order ▾

☐

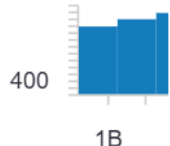
Reverse order

Feature search (regex enabled)

Features: ☒ int(2) ☒ float(4)

Numeric Features (6)

count	missing	mean	std dev	zeros	min	median	max	custom	Chart to show Standard ☐ log ☐ expar	
tpep_pickup_datetime	21.9k	0%	1.45B	1.50M	0%	1.45B	1.45B	1.46B	data type: timestamp min: 2016-01-01T00:04:30Z	



You can use dbutils as well... count, missing, mean, std dev, zeros, min, median,max

Data Profiling

▶

✓

12:32 AM (2s)

4

Python

🗑️

🔍

⋮

```
display(df.describe())
```

▶ (2) Spark Jobs

Table ▼ +

🔍 🔍 🗑️

	summary	trip_distance	fare_amount	pickup_zip	dropoff_zip
1	count	21932	21932	21932	21932
2	mean	2.8528291993434256	12.348726974284153	10143.6594929782...	10174.823454313331
3	stddev	3.4399536210623505	10.327887859233348	343.085358777764...	403.1885553811378
4	min	0.0	-8.0	7002	7002
5	max	30.6	275.0	11436	11694

⬇️

5 rows | 2.32 seconds runtime

Refreshed 13 minutes ago

Or dataframe.describe... count, missing, mean, std dev, zeros, min, median,max

Implementation for Lakehouse

- So we need to have traceability and the capability to move back into the history of data loads.
- We also want to enable change data capture...



Leverage Delta Tables

Delta Lake / Delta Tables

- Delta Lake is the optimized storage layer that provides the foundation for tables in a lakehouse on Databricks. Delta Lake is open source software that extends Parquet data files with a file-based transaction log for ACID transactions and scalable metadata handling.
- Delta Lake is fully compatible with Apache Spark APIs, and was developed for tight integration with Structured Streaming, allowing you to easily use a single copy of data for both batch and streaming operations and providing incremental processing at scale.

Src: <https://docs.databricks.com/en/delta/index.html>

Delta Lake / Delta Tables

- Delta Lake is the default format for all operations on Databricks. Unless otherwise specified, all tables on Databricks are Delta tables. Databricks originally developed the Delta Lake protocol and continues to actively contribute to the open source project. Many of the optimizations and products in the Databricks platform build upon the guarantees provided by Apache Spark and Delta Lake.

Delta Tables

- Traceability and Historical Data Access
 - Delta Lake provides powerful features for traceability and historical data access:
 - Time Travel: Delta Lake allows you to query previous snapshots of your data using the "versionAsOf" option. This enables you to:
 - Access historical data states
 - Audit data changes over time
 - Rollback to previous versions if needed

Src: <https://docs.delta.io/latest/delta-intro.html>

Delta Tables & CDC

- Transaction Log: Delta Lake maintains a transaction log that records all changes made to a table¹. This log is crucial for:

- Ensuring data lineage
- Enabling time travel functionality
- Providing a complete audit trail of data modifications

- CDF is associated with a Delta table and is stored in TBLPROPERTIES. To Enable CDF for a table, you need to set the TBLPROPERTIES `delta.enableChangeDataFeed` to true.

Delta Tables & CDC

Cmd 3

```
1 CREATE TABLE IF NOT EXISTS LakehouseCookbook.Watermarks (  
2   schemaName string,  
3   tableName string,  
4   watermarkType string,  
5   timestampWatermark TIMESTAMP,  
6   integerWatermark INT,  
7   bigIntWatermark BIGINT,  
8   stringWatermark string  
9 ) TBLPROPERTIES (delta.enableChangeDataFeed = true)
```

▶ (4) Spark Jobs

OK

Command took 9.85 seconds -- by alan.dennis@gmail.com at 7/1/2023, 3:11:59 PM on Power User Cluster

Cmd 4

```
1 DESCRIBE HISTORY lakehousecookbook.Watermarks
```

▶ (1) Spark Jobs

Table ▾ +

	version	timestamp	userId	userName	operation	operationParameters
1	0	2023-07-01T19:12:02.000+0000	2270508095937838	alan.dennis@gmail.com	CREATE TABLE	▶ {"isManaged": "true", "descrip

Incremental Ingestion Example

- There are several types of ingestion, but incremental ingestion is the preferred mechanism with the Lakehouse architecture.
- Lakehouses consume data from enterprise systems (CRMs/ERPs...etc).

EmPIId	FirstName	LastName	Address1	CreatedOn	ModifiedOn
1	Leonardo	da Vinci	123 First St.	12/31/2024	12/31/2024
2	Napoleon	Bonaparte	234 Second St.	12/31/2024	12/31/2024
3	Charles	Darwin	345 Third St.	12/31/2024	12/31/2024
4	Albert	Einstein	456 Fourth St.	12/31/2024	12/31/2024
5	Thomas	Jefferson	678 Fifth St.	12/31/2024	12/31/2024

During the initial ingestion of the employee table, we would load all records.

Incremental Ingestion Example

- If an employee moves to a new residence, we update that employee's record with their new address, also changing the revision timestamp. When we ingest records from the employee database, we would only get those records which have changed.
- Albert decided to move to a different location. We update the address in the employee database table along with the modified date. Often modification timestamps are system managed

EmPIId	FirstName	LastName	Address1	CreatedOn	ModifiedOn
1	Leonardo	da Vinci	123 First St.	12/31/2024	12/31/2024
2	Napoleon	Bonaparte	234 Second St.	12/31/2024	12/31/2024
3	Charles	Darwin	345 Third St.	12/31/2024	12/31/2024
4	Albert	Einstein	160 Spear St	12/31/2024	5/1/2025
5	Thomas	Jefferson	678 Fifth St.	12/31/2024	12/31/2024

Src: Databricks Lakehouse Platform Cookbook Dr. Alan Dennis

Incremental Ingestion Example

- When extracting data from this table into our Lakehouse, only records with modification dates greater than 12/31/2024 will be extracted

EmPIId	FirstName	LastName	Address1	CreatedOn	ModifiedOn
4	Albert	Einstein	160 Spear St	12/31/2024	5/1/2025

What Happens to the Bronze layer?????

Incremental Ingestion Example

- The Bronze table will contain all changes made to the source table.

EmPIId	FirstName	LastName	Address1	CreatedOn	ModifiedOn
1	Leonardo	da Vinci	123 First St.	12/31/2024	12/31/2024
2	Napoleon	Bonaparte	234 Second St.	12/31/2024	12/31/2024
3	Charles	Darwin	345 Third St.	12/31/2024	12/31/2024
4	Albert	Einstein	160 Spear St	12/31/2024	12/31/2024
5	Thomas	Jefferson	678 Fifth St.	12/31/2024	12/31/2024
4	Albert	Einstein	160 Spear St	12/31/2024	5/1/2025

Many struggle to understand that we only append to the Bronze table. The primary reason for this design decision is to preserve the history of changes.

Full vs. Incremental Load

Full Load:

- Copies the entire dataset from the source to the target, replacing all existing data.
- Simpler to implement but more resource-intensive and time-consuming, especially for large datasets.
- Ensures data integrity but may be impractical for frequent updates or large volumes of data.

Incremental Load:

- Only loads new or updated records since the last load.
- More efficient and faster, especially for large datasets with frequent updates.
- Requires a mechanism to track changes, such as timestamps or version numbers.
- More complex to implement but saves time and resources.

Creating vs. Replacing a table

- Creating a new table without replacing it does not preserve the history, while replacing a table using the CREATE OR REPLACE TABLE statement keeps the history intact.
- **Creating a new table:** If you drop a table and create a new one with the same name, the history is lost. This is because dropping a table removes it from the metastore and deletes the underlying data (for managed tables) or removes the table definition (for external tables).

Creating vs. Replacing a table

Replacing a table: Using `CREATE OR REPLACE TABLE` preserves the table's history and identity. This approach offers several advantages:

- Table contents are replaced, but the table identity is maintained.
- The table history is retained, allowing you to revert to earlier versions using the `RESTORE` command.
- The operation is atomic, ensuring the table always exists during the replacement process.
- Concurrent queries can continue without interruption, accessing either the old or new version of the table as needed.

THE GEORGE
WASHINGTON
UNIVERSITY

WASHINGTON, DC

APIs: Rest and GraphQL

What is an API?

An API (Application Programming Interface) is a defined way for two software systems to communicate (with or without a network).

It specifies:

- What requests can be made
- What operations are available
- What data is required
- What format the response will have
- What outputs will be returned

APIs can exist:

- Between applications over the internet
- Between services in a distributed system
- On your local machine
- Inside a software library (like Pandas – technically an API with a set of functions, classes, methods, and data structures that you use to interact with the underlying pandas library).

In data engineering, APIs are commonly used to:

- Ingest external data
- Integrate services
- Expose processed data to other systems

Examples of APIs

Library APIs

- Functions and classes exposed by a software library that your program can call directly.
Examples include data processing libraries, machine learning frameworks, and visualization tools.

Local Service APIs

- APIs that run on your own machine and are accessed through a local network address.
Often used in development environments and microservice architectures.

Remote Web APIs

- APIs hosted on remote servers and accessed over the internet using HTTP.
Commonly used to retrieve external data or interact with cloud platforms.

Operating System APIs

- Interfaces that allow software to interact with the operating system.
Used for file operations, memory management, networking, and system processes.
- An API is defined by the interaction contract, not by whether it runs locally or remotely.

How APIs Work

APIs typically follow a client–server model:

- The client sends a request
- The server processes the request
- The server returns a structured response (usually JSON or similar)

```
GET https://api.example.com/users/123
```

What is REST?

REST (Representational State Transfer) is an architectural style for designing web APIs.

REST APIs:

- Use standard HTTP methods (GET, POST, PUT, DELETE)
- Treat data as resources identified by URLs
- Are stateless (each request contains all necessary information)
- Commonly return JSON responses

REST is the most widely used API style in industry.

HTTP Methods in REST

REST APIs use HTTP methods to define actions:

- **GET** → Retrieve data
- **POST** → Create new data
- **PUT** → Update existing data
- **DELETE** → Remove data

```
GET /users/123      → Retrieve user  
POST /users         → Create user  
DELETE /users/123   → Delete user
```

Example REST Response

- Most APIs return JSON (JavaScript Object Notation):
JSON
- Uses key-value pairs
- Is lightweight
- Is easily parsed by programming languages
- Is dominant format in modern APIs

```
{  
  "user_id": 123,  
  "name": "Alice",  
  "email": "alice@email.com"  
}
```


HTTP Status Codes

Every API response includes a status code indicating the result.

Common codes:

- **200** → Success
- **400** → Bad request
- **401** → Unauthorized
- **404** → Resource not found
- **429** → Too many requests (rate limit exceeded)
- **500** → Server error

Production data systems must handle these errors gracefully.

What is GraphQL?

GraphQL is a query language for APIs that allows clients to request exactly the data they need.

Key differences from REST:

- Uses a single endpoint
- Clients specify the structure of the response
- Reduces over-fetching and under-fetching of data
- Example query:

```
{  
  user(id: 123) {  
    name  
    email  
  }  
}
```

GraphQL is often used in complex front-end applications.

REST vs. GraphQL

REST

- Multiple endpoints
- Server defines response structure
- Can return more data than needed

GraphQL

- Single endpoint
- Client defines response structure
- More flexible but more complex

REST is simpler and more common in data pipelines.
GraphQL is common in application development.

REST vs. GraphQL Continued

HTTP = the delivery truck
REST or GraphQL = the language inside the package

- Both REST and GraphQL:
- Use HTTP
- Use URLs
- Return JSON
- Have status codes

```
GET /users  
GET /users/123  
POST /users  
DELETE /users/123
```

REST

```
POST /graphql
```

GraphQL

Other API Architectures

- While REST and GraphQL are most common, other API styles exist:

<u>Type</u>	<u>Description</u>	<u>Common Use Case</u>
SOAP	XML-based protocol with strict standards	Enterprise legacy systems
gRPC	High-performance binary protocol	Microservices, low-latency systems
WebSockets	Persistent connection for real-time communication	Streaming, chat systems
Webhooks	Server pushes data when events occur	Event-driven integrations

Modern data engineering primarily uses:

- REST
- gRPC (in distributed systems)
- Webhooks for event ingestion

Pagination in APIs – convert to table

- APIs rarely return all data at once because datasets can be very large.
- Pagination limits the amount of data returned per request.
- Data engineers must loop through pages to collect full datasets.

Common pagination strategies:

<u>Pagination Type</u>	<u>How It Works</u>	<u>Example</u>	<u>Strengths</u>	<u>Weaknesses</u>
Offset-Based	Skips a number of records before returning results	<code>/users?limit=100&offset=200</code>	Simple to understand	Slow for large datasets; unstable if data changes
Page-Based	Requests data by page number	<code>/users?page=3&size=100</code>	Easy for UI applications	Similar scalability issues as offset
Cursor-Based	Uses a pointer to the last returned record	<code>/users?cursor=abc123</code>	Scales well; stable for dynamic data	Slightly more complex implementation

Pagination in AWS

- AWS APIs use pagination because service results can be very large.
- Conceptually, it is the same as standard pagination, but uses tokens instead of page numbers.
- Example (AWS SDK concept):

```
response = s3.list_objects_v2(Bucket="my-bucket", MaxKeys=1000)

if response["IsTruncated"]:
    next_token = response["NextContinuationToken"]
```

- Key idea:
You must continue calling the API until no continuation token is returned.
- This is the same core concept as cursor-based pagination.

Authentication – more info

Authentication ensures that only authorized users or systems can access an API.

<u>Method</u>	<u>How It Works</u>	<u>Security Level</u>	<u>Common Use</u>
API Key	Static key sent with request header	Low–Moderate	Internal tools, simple services
Basic Auth	Username and password encoded in request	Low	Legacy systems
OAuth 2.0	Token issued after authentication flow	High	Enterprise and SaaS platforms
Bearer Token (JWT)	Encoded token containing claims	High	Microservices and web apps

- Tokens expire
- Secrets must not be hardcoded
- Credential rotation must be handled
- Unauthorized responses (401) must trigger re-authentication

Data pipelines must securely manage and refresh credentials.

Rate Limiting and Retry Strategies

Most APIs limit how many requests can be made in a time window.

Example limits:

- 100 requests per minute
- 10,000 requests per day

If exceeded, the API returns something along the lines of
429 – Too Many Requests

Data engineering systems must implement:

- Retry logic
- Exponential backoff
- Respect for Retry-After headers
- Logging of failed attempts

APIs in Data Engineering

APIs are used to:

- Pull data from SaaS platforms
- Ingest third-party datasets
- Connect microservices
- Trigger workflows
- Example ingestion pattern:

API -> Raw Storage -> Processing -> Data Warehouse -> Analytics

Leveraging/understanding APIs is foundational before learning streaming and distributed processing.

Microservices (Services that are...micro)

A microservice is a small, independent application that performs a specific business or technical function and communicates with other services through APIs. **Doesn't necessarily have to be a comprehensive software tool.**

Characteristics

- Single responsibility
- Independently deployable
- Scalable on its own
- Loosely coupled
- Communicates via APIs or messaging systems

Why Use Them?

Monolithic Application:

- One large application performs everything

Microservices Architecture:

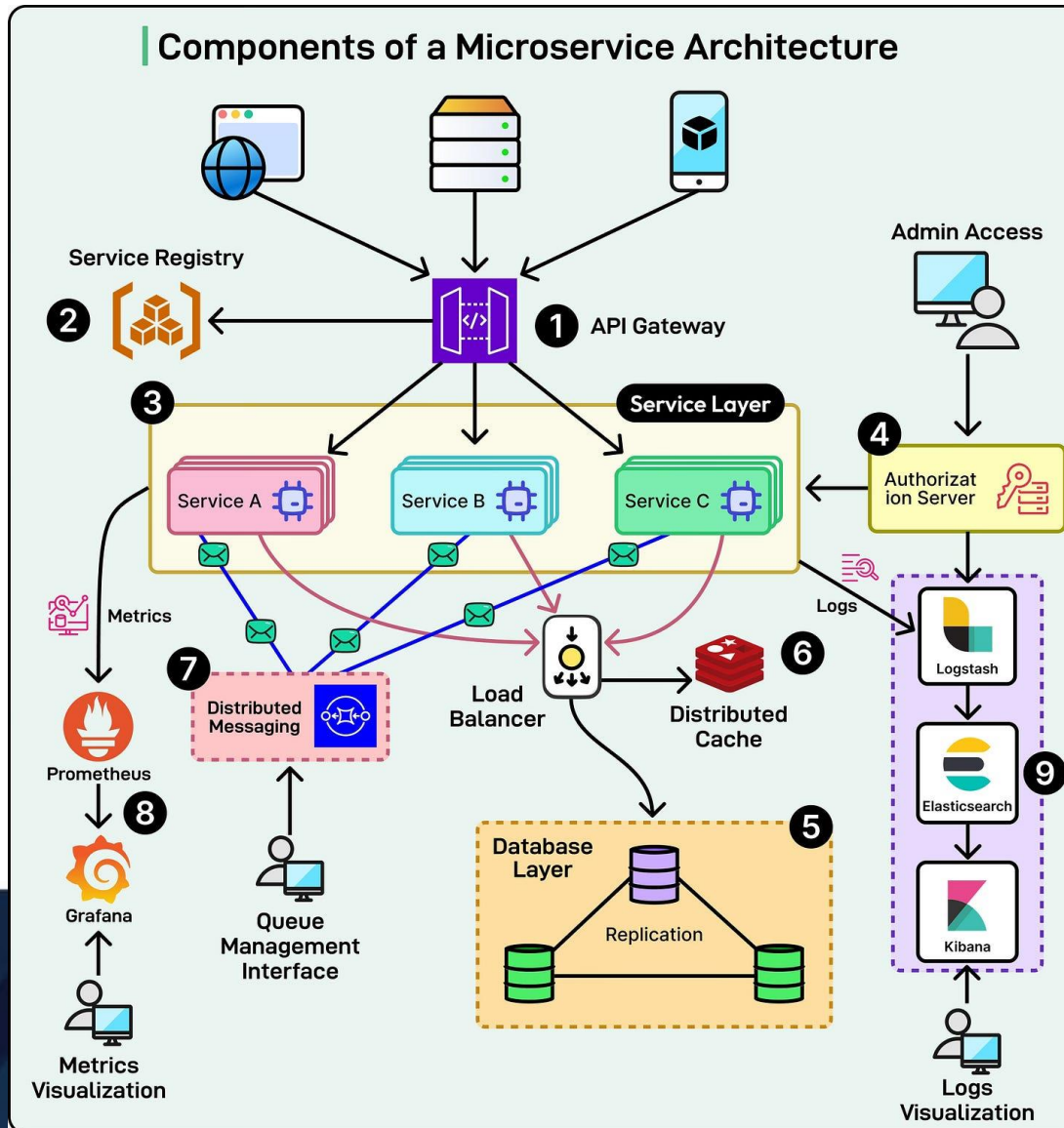
- Separate services handle:
 - Data ingestion
 - Data transformation
 - Data storage
 - Model inference
 - Monitoring
 - User interfaces

Example

- Customer uploads file -> Ingestion Service, Transformation Service, Feature Engineering Service, Model Inference Service, Results API

In a modern data platform, each stage of the Bronze → Silver → Gold pipeline may be implemented by one or more microservices. Some may be simple Python scripts, while others may be large distributed systems such as Spark jobs running in Databricks

Microservices Diagram Example

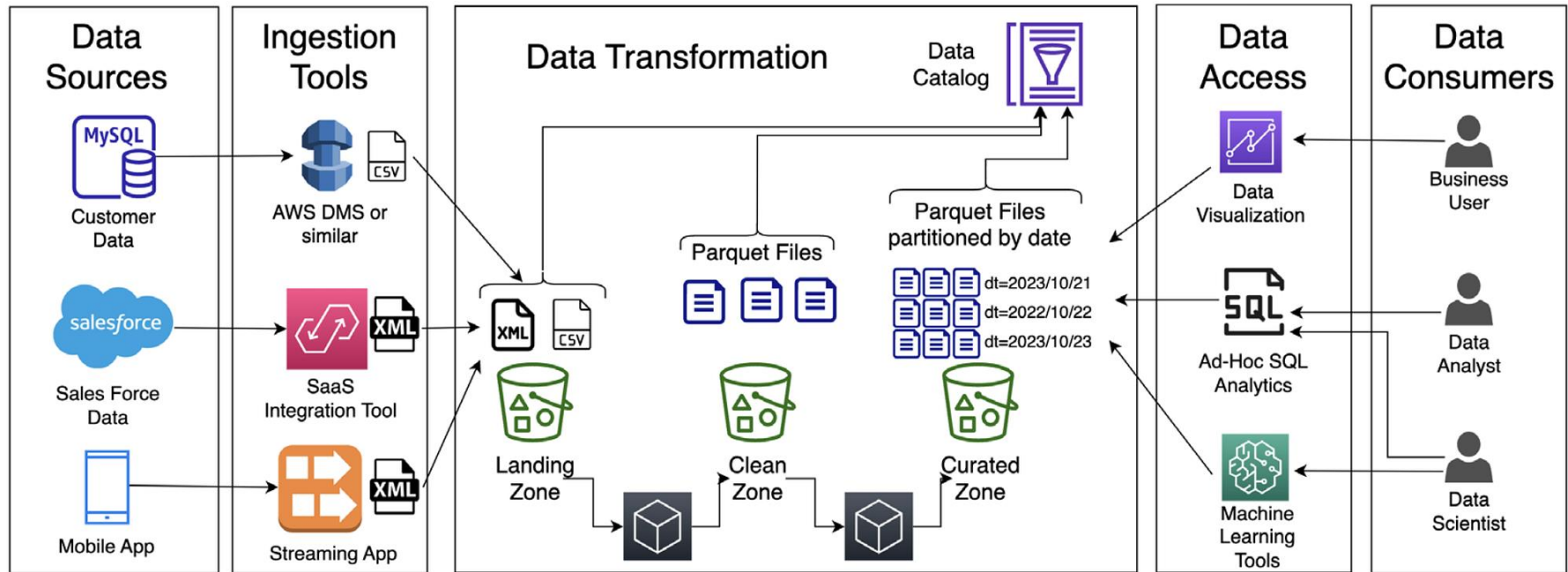


Source

Common Data Engineering Microservices

Microservice	Purpose	Input Example	Output Example
Data Ingestion Service	Collect data from external sources	API response, CSV file, database record	Raw JSON stored in Bronze layer
Data Validation Service	Verify schema and data quality	Raw customer record	Validated record with errors flagged
Transformation Service	Clean and standardize data	Raw sales transaction	Structured Silver dataset
Data Enrichment Service	Add context from other sources	Customer ID	Customer profile with demographics
Storage Service	Persist data to data lake/warehouse	Processed dataset	Delta table, Parquet file, SQL table
Metadata Service	Track lineage and governance	Dataset information	Catalog entry and lineage record
Orchestration Service	Manage workflow execution	Pipeline trigger	Scheduled execution of downstream tasks

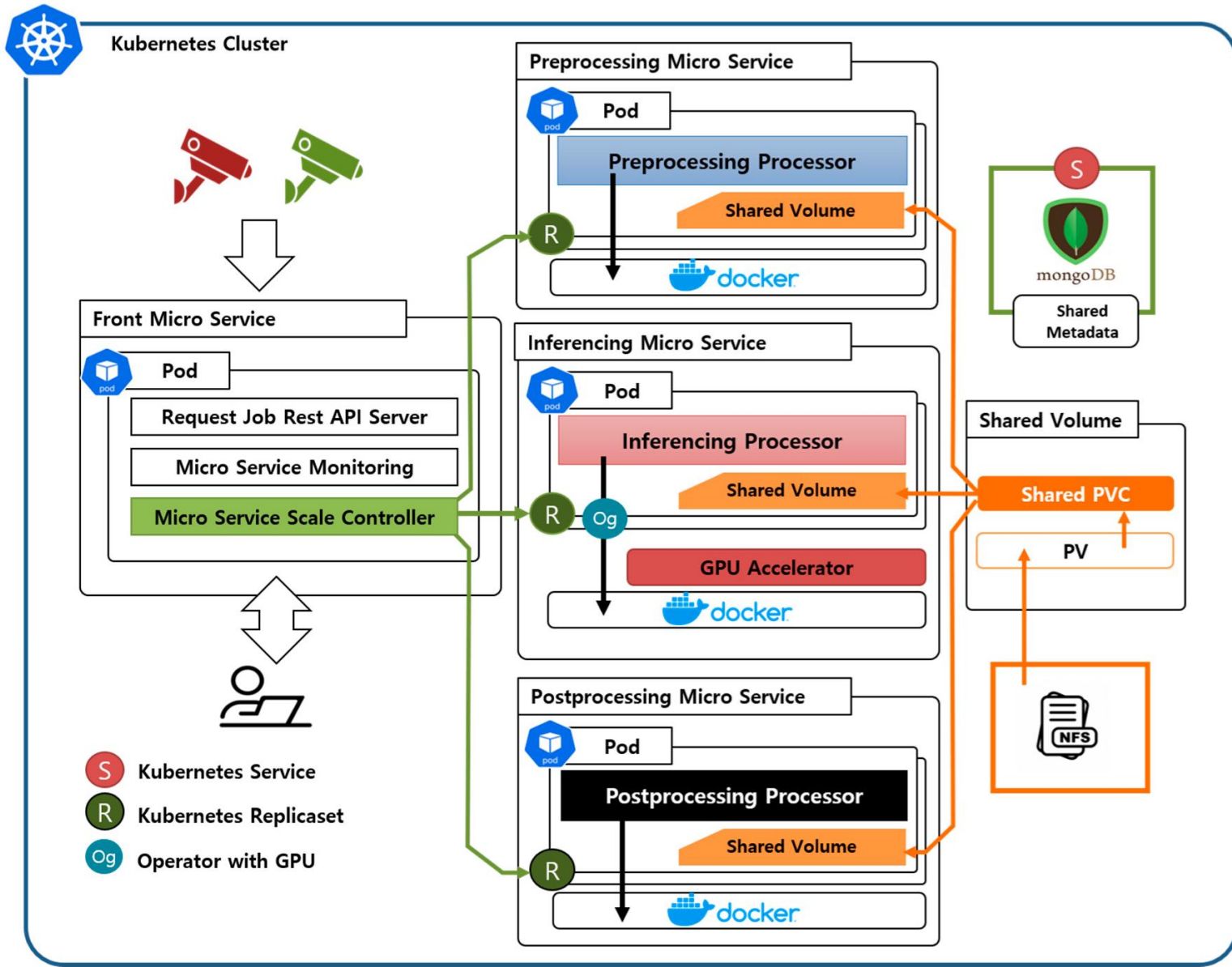
Data Engineering Microservices Example Diagram



[Source](#)

Common AI/ML Microservices

Microservice	Purpose	Input Example	Output Example
Feature Engineering Service	Generate model features	Customer transaction history	Feature vector
Model Training Service	Train machine learning models	Training dataset	Trained model artifact
Model Registry Service	Store and version models	Trained model	Versioned model record
Model Inference Service	Generate predictions	Feature vector	Prediction and confidence score
Monitoring Service	Track performance and drift	Predictions and actual outcomes	Performance metrics
Feedback Service	Collect production results	User actions, labels	Retraining dataset
Retraining Service	Update model with new data	New training data	Updated model version



Source

THE GEORGE
WASHINGTON
UNIVERSITY

WASHINGTON, DC

API Demo

THE GEORGE
WASHINGTON
UNIVERSITY

WASHINGTON, DC

What to work on this week

What To Work On This Week

- Homework #4 (Hands-on Assignment)
- Reading: Chapters 7 of Data Engineering Fundamentals on O'Reilly
 - Remember to check the syllabus for assigned readings